M4 Traffic and AI — Module Proposal

Nov 1, 2026 · @Taha

1. Summary and scope

M4 is a deterministic software library on the main computer. It finds routes and travel times on the road network, moves the NPC cars, and drives the AI bots by producing the same controls that a phone sends. It has no network, rendering or physics code of its own: everything goes in and out through the data structures in Sections 3 and 4, so M4 can be built and tested without any other module.

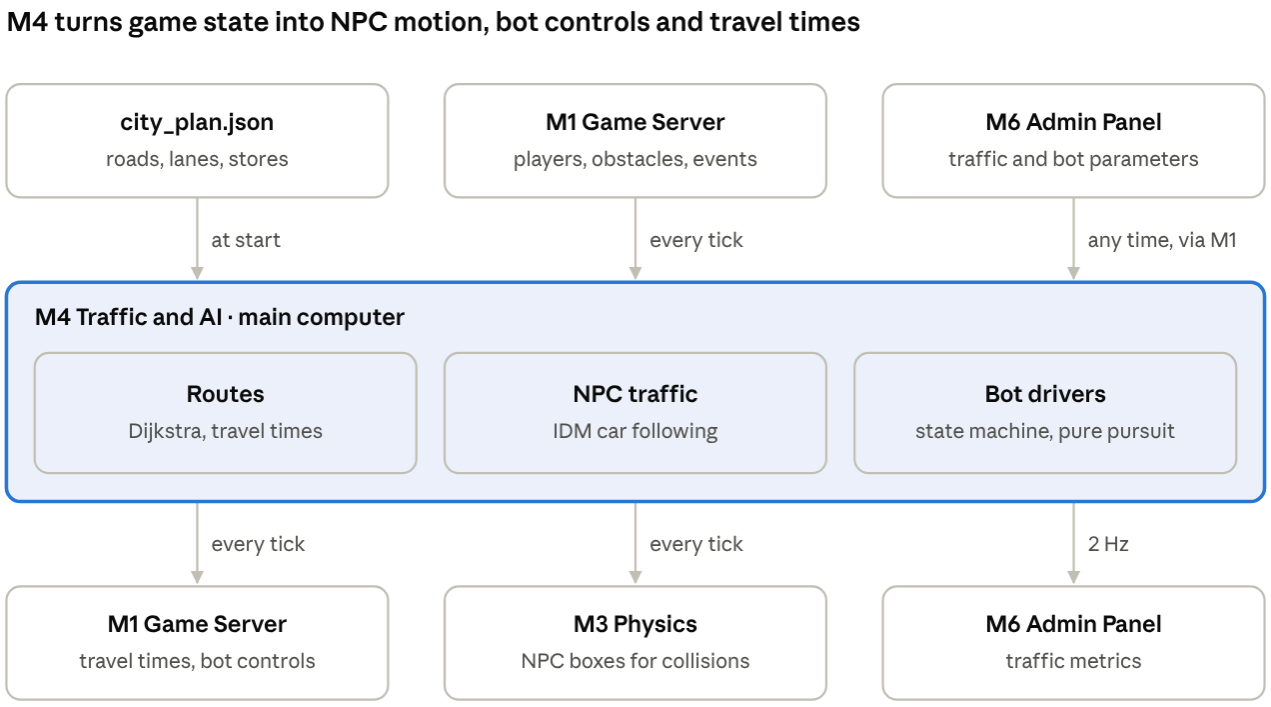
Item	Detail
Members	Hasan Alperen Elmas, Batuhan Yanar, Muhib Taha Boy
Runs on	Main computer, inside the game server process; called by M1 once per server tick
In scope	Road graph built from the shared city plan, shortest paths, travel-time table, NPC traffic, bot drivers, traffic metrics
Out of scope	Choosing the delivery point (M1), motorcycle physics and collisions (M3), drawing cars (M5), networking (M2)

Three design rules keep M4 independent and stable:

- **Pure tick function.** The same inputs and the same random seed always give the same outputs. Any bug can be replayed from a log file.
- **Bots are players.** A bot sends exactly the same control message as a phone. M3 does not know if a motorcycle is a human or a bot, so the game stays fair and there is only one physics path to test.
- **Language-neutral contract.** All inputs and outputs are plain JSON-serializable objects. M4 can be called as a function inside the server or through a local socket if the server language changes.

2. Position in the system

M4 never talks to the network or the board directly. M1 feeds it the game state every tick and forwards its outputs; M3 reads the NPC cars for collisions, and collisions come back to M4 inside the next tick input.



M4 in the system · 3 inputs, 3 outputs

Because every arrow is a data structure from Section 4, each neighbour can be replaced by a mock during development (Section 8.1).

3. Interface contract: inputs and outputs

M4 exposes six calls. M1 calls `tick` once per server tick; every other call is either at start-up or on demand. Names below are language-neutral; Section 5 explains the language choice.

```
init(cityPlan, params, seed)        -> void            // once, at game start
setParams(partialParams)            -> void            // any time; applied
at next tick
tick(TickInput)                     -> TickOutput      // every server tick
(30 Hz default)
getTravelTimes(teamId)              -> TravelTimeTable // M1 delivery-point
selection
getRoute(fromPos, toPos)            -> Route           // bots, debug view,
optional
getMetrics()                        -> TrafficMetrics  // M6 graphs, polled
at 2 Hz
```

Inputs

Input	From	When	Content
City plan	Shared file `city_plan.json`	`init`	Intersections, road segments, lanes, stores, delivery candidates (Section 4.1)
Parameters	M6, passed through M1	Any time	NPC count and speed, obstacle effects, bot skill (Section 7)
Players	M1 game state	Every tick	Position, heading, speed, team, bot flag, fallen flag, current target
Obstacles	M1 game state	Every tick, with a version number	Segment, type, lane and position on the segment
Events	M3 through M1	Every tick	Motorcycle–NPC collisions, earthquake start

Outputs

Output	To	When	Content
NPC cars	M3 (collisions), M1 to M2 to M5 (drawing)	Every tick	ID, position, heading, speed, lane, size
Bot controls	M1, then M3	Every tick, one per bot	Same message as a phone: throttle, brake, steer, deliver

Output	To	When	Content
Travel-time table	M1	On request; recomputed when obstacles change	Seconds from a team store to every delivery candidate
M4 events	M1, M6	When they happen	Bot replanned, bot stuck, NPC removed
Traffic metrics	M6	2 Hz	NPC count, mean NPC speed, bot route changes

What other modules must agree to

These four items are shared contracts. They are fixed in this proposal and must match the M1, M2, M3 and M5 proposals.

1. **City plan schema** (Section 4.1): one file, one version number, used by M1, M3, M4, M5 and M8.
2. **World coordinates**: metres, origin at the north-west corner intersection, x to the east, z to the south, y up. Heading in degrees clockwise from north.
3. **Control message**: `throttle` and `brake` in [0, 1], `steer` in [−1, 1], `deliver` true/false. Phone buttons send only 0, 1 and −1; bots use the same values.
4. **Obstacle model**: four types (`heavy_traffic` , `road_work` , `accident` , `puddle`) placed on a segment ID, with optional lane and position.

4. Data formats with examples

All examples are the exact JSON that crosses the M4 boundary. Units are always metres, seconds and degrees; `null` means "unreachable" or "not set".

4.1 Shared city plan and ID scheme

The city has 6 × 6 = 36 intersections and 60 road segments (30 east–west, 30 north–south). IDs are fixed by formula, so every module can compute them without a lookup table:

- Intersection `n_r_c` (row r, column c, 0–5): `id = 6r + c` (0–35)
- East–west segment `h_r_c` between `n_r_c` and `n_r_(c+1)` : `id = 5r + c` (0–29)
- North–south segment `v_r_c` between `n_r_c` and `n_(r+1)_c` : `id = 30 + 6r + c` (30–59)

```
{
  "version": "1.0.0",
  "world": { "size_m": 500, "origin": "n_0_0", "axes": "x east, z south, y
up" },
  "grid": { "lines": 6, "pitch_m": 100, "lanes_per_direction": 2,
"lane_width_m": 3.5 },
  "nodes": [ { "id": 0, "name": "n_0_0", "x": 0, "z": 0 } ],
  "segments": [ { "id": 31, "name": "v_0_1", "a": 1, "b": 7, "length_m": 100
} ],
  "stores": [ { "team": 0, "color": "blue", "node": 0, "pickup": { "x": 18,
"z": 18, "radius_m": 10 } } ],
  "delivery_candidates": [ { "id": 0, "segment": 31, "t": 0.5, "side": "E" }
]
}
```

The board (M8) uses the same file with normalised coordinates `u = x / size_m`, `v = z / size_m`. The values 500 m and 100 m are proposals; M3 and M5 can change `size_m` without any change in M4.

4.2 Tick input (M1 → M4)

```
{
  "tick": 1824, "t": 60.80, "dt": 0.0333,
  "obstacles_version": 7,
  "obstacles": [
    { "id": 3, "segment": 12, "type": "road_work" },
    { "id": 4, "segment": 41, "type": "accident", "lane": 1, "t": 0.4 },
    { "id": 5, "segment": 20, "type": "heavy_traffic" }
  ],
  "players": [
    { "id": 2, "team": 1, "bot": true, "x": 212.4, "z": 98.1, "heading_deg":
87.5,
      "speed_mps": 11.2, "phase": "to_delivery", "fallen": false,
      "target": { "x": 350.0, "z": 152.0, "radius_m": 6 } },
    { "id": 5, "team": 3, "bot": false, "x": 410.0, "z": 300.2,
"heading_deg": 180.0,
      "speed_mps": 7.9, "phase": "to_store", "fallen": false }
  ],
  "events": [ { "type": "npc_hit", "npc": 17, "player": 5 } ]
}
```

4.3 Tick output (M4 → M1, M3)

```
{
  "tick": 1824,
  "npc": [
    { "id": 17, "x": 405.1, "z": 296.0, "heading_deg": 0.0, "speed_mps": 0.0,
      "segment": 51, "lane": 0, "len_m": 4.2, "wid_m": 1.8 }
  ],
  "bot_controls": [
    { "player": 2, "seq": 912, "throttle": 1, "brake": 0, "steer": 0,
"deliver": false }
  ],
  "events": [ { "type": "bot_replanned", "player": 2, "reason":
"obstacles_changed" } ]
}
```

4.4 Travel-time table (M4 → M1)

```
{ "team": 1, "obstacles_version": 7, "computed_at_tick": 1790,
  "times_s": { "0": 41.2, "1": 38.7, "17": null } }
```

4.5 Parameter update (M6 → M1 → M4)

```
{ "npc.count": 40, "npc.max_speed_mps": 10,
  "heavy_traffic.speed_factor": 0.4, "bot.skill": 0.6 }
```

Unknown keys are rejected with an error message to M6; missing keys keep their old value. Every message carries `tick` or a version number, so an old message can never overwrite a newer one.

5. Technology choices and alternatives

Recommendation: TypeScript on Node.js, with no third-party algorithm libraries. The server language is not fixed by the group yet, so M4 is designed as a pure library that can be ported in about a week if the group chooses another language.

5.1 Language

Option	Strengths	Weaknesses	Difficulty	Stability
TypeScript / Node.js (recommended)	Same language as the Three.js phone client (M5); city plan types and helpers are shared; good WebSocket libraries for M2; strict types catch contract errors at compile time	Single thread; heavy physics may need a worker thread	Low	High; Node LTS releases are supported for 30 months
Python	Easy to write; good for plotting and test analysis	Slower loops and garbage-collection pauses at 30–60 Hz; no code shared with the browser client	Low	Medium; timing jitter under load
C# / .NET	Fast and strongly typed	No code shared with the browser client; fewer team members know it	Medium	High

At the expected load (⩽ 80 NPC cars, ⩽ 8 bots, a 36-node graph) all three are fast enough. The deciding factors are code sharing with M5 and one language for the whole server.

5.2 Libraries and tools

Need	Choice	Why	Alternative
Shortest path	Own Dijkstra, about 60 lines	The graph has 36 nodes; no dependency to break	`ngraph.path`
Random numbers	Own seeded generator (Mulberry32)	Same seed gives the same game; `Math.random` cannot be seeded	`seedrandom`

Need	Choice	Why	Alternative
Input checking	`zod` schemas	Rejects a malformed message with a clear error instead of a crash	Hand-written checks
Unit tests	`vitest`	Fast, TypeScript native	`jest`
Property tests	`fast-check`	Thousands of random obstacle layouts per test run	Hand-made random loops
Debug view	HTML canvas page, top-down 2D	Shows the graph, cars and bot paths live; no 3D needed	Print logs only

5.3 Algorithm choices

Problem	Choice	Rejected alternative and reason
Route finding	Dijkstra on the segment graph	A* gives no gain on 36 nodes and adds a heuristic that can be wrong
NPC following	Intelligent Driver Model (IDM)	Fixed speed with stop-if-blocked: cars jump and pile up, looks unnatural
NPC at intersections	One car inside an intersection at a time, plus a stuck timer	Traffic lights: more states, more deadlock cases, little gameplay value
Bot steering	Pure pursuit on the lane centre line	PID on heading error: oscillates at high speed and needs more tuning
Bot decisions	Finite state machine	Behaviour trees or learning: too large for 6 states

6. How it works

Every tick M4 runs four steps in a fixed order: update the road costs, move the NPC cars, decide the bot controls, then update the metrics. A fixed order is what makes the tick deterministic.

6.1 Road graph

At `init`, M4 builds a directed graph from the city plan: 36 nodes and 120 directed edges (each of the 60 segments in both directions). Each directed edge stores its lanes, its length and a cost in seconds. Turns at an intersection add a turn penalty (about 1.5 s for left and right turns, 4 s for a U-turn), because a motorcycle must slow down to turn.

6.2 Routes and travel times

The cost of an edge is its length divided by the expected speed on it:

```
c_e = \frac{L_e}{v_{free} \cdot f_{type} \cdot f_{density}} + p_{turn}
```

- `f_type` comes from the obstacle on the edge: 1.0 for no obstacle, 0.4 for heavy traffic, 0.5 for an accident, 1.0 for a puddle (plus a small risk penalty). Road work closes the edge (cost = infinity).
- `f_density` falls as the number of NPC cars on the edge rises.

When `obstacles_version` changes, M4 runs Dijkstra once from every node (36 runs on 120 edges, well under 1 ms) and stores the full table. The travel time to a delivery candidate at position `t` on segment AB is the smaller of "reach A, then drive t·L" and "reach B, then drive (1−t)·L". M1 reads this table to pick fair near or far points for each team.

6.3 NPC traffic

Each NPC car lives on one lane of one directed edge and has a position along it. Its acceleration follows the Intelligent Driver Model:

```
a = a_{max}\left[1 - \left(\frac{v}{v_0}\right)^4 - \left(\frac{s^*}
{s}\right)^2\right], \quad s^* = s_0 + vT + \frac{v\,\Delta v}{2\sqrt{a_{max}
b}}
```

The "car ahead" can be another NPC, a motorcycle in the same lane, a closed lane (accident) or a busy intersection. So cars slow down smoothly behind each other and behind players. Further rules:

1. At an intersection a car picks a random allowed exit (no closed roads, no U-turn unless it is a dead end) and enters only when the intersection is free.
2. A car changes lane only when its own lane is closed ahead.
3. A spawner keeps the total count at `npc.count`. Heavy-traffic segments get more cars and a lower `v0`. New cars appear only where no player is closer than 30 m, so cars never pop up in front of a player.
4. After a collision event from M3, the car stops for 2 s, then continues.

5. A car that has not moved for 10 s is removed and spawned again elsewhere. This rule makes deadlock impossible.

6.4 Bots

Each bot is a finite state machine. Its states follow the delivery cycle of the game:

1. **To store**: drive the route to the team store.
2. **Pickup**: wait inside the store area until M1 marks the package as taken.
3. **To delivery**: drive the route to the delivery point.
4. **Deliver**: when inside the delivery radius and slow enough, send `deliver: true`.
5. **Fallen**: no controls while M1 reports `fallen: true`.
6. **Recover**: if speed stays under 0.5 m/s for 3 s, steer fully to one side with low throttle (no reverse exists) and replan.

The route is a list of lane centre lines and turn arcs. Steering uses pure pursuit: the bot aims at a point about `0.6 s × speed + 4 m` ahead on its path. Speed control slows down before turns and keeps an IDM gap to the car in front, or changes lane if the other lane is free. A bot replans when the obstacles change or when it is more than 8 m off its path.

One `bot.skill` value (0 to 1) sets the bot's top speed, reaction delay (150–600 ms), steering noise and lane-change courage. The admin can therefore make bots weaker or stronger than the human players.

7. Parameters and calibration

All M4 parameters can be changed live from the admin panel and take effect on the next tick. Defaults below are starting values; the calibration steps after the table fix the final ones.

Key	Default	Range	Effect
`npc.count`	30	0–80	Total NPC cars in the city
`npc.max_speed_mps`	10	4–16	Free-road speed of NPC cars
`npc.segment_density`	none	0–10 per segment	Admin override for one road
`npc.idm.T_s` / `s0_m`	1.2 / 2.0	0.8–2.0 / 1–4	Time gap and minimum gap between cars

Key	Default	Range	Effect
`heavy_traffic.speed_factor`	0.4	0.1–1.0	NPC speed on heavy-traffic roads
`heavy_traffic.density_factor`	3	1–5	Extra NPC cars on heavy-traffic roads
`accident.speed_factor`	0.5	0.1–1.0	Speed in the slow zone of an accident
`route.turn_penalty_s`	1.5	0–5	Extra time per turn in travel times
`route.puddle_penalty_s`	1.0	0–10	How much routes avoid puddles
`bot.skill`	0.6	0–1	Bot top speed, reaction time, steering noise
`bot.replan_period_s`	1.0	0.2–5	How often bots check for a better route
`seed`	random	any integer	Fixed seed for repeatable test games

Calibration steps

1. **Travel-time model.** Run bots (skill 1.0) between all 4 stores and all delivery candidates with no NPCs, then with NPCs. Fit `route.turn_penalty_s` and the density factor by least squares so that predicted and measured times agree. Target: mean absolute error ⩽ 15%. This is what makes delivery points fair for all four teams.
2. **NPC speed against players.** With the motorcycle physics from M3, set `npc.max_speed_mps` to about 50–60% of the motorcycle top speed, so players can pass cars but must plan it.
3. **Bot difficulty.** In play tests, record human delivery times. Choose the `bot.skill` that gives a bot delivery time within ±15% of the human median, and store it as the default.
4. **Heavy-traffic feel.** Tune `heavy_traffic.speed_factor` and `density_factor` until a heavy-traffic road is clearly slower (at least 1.5× the travel time) but still passable.

Each calibration run is a script that writes a CSV file, so results can be repeated and shown in the final report.

8. Standalone testing

M4 is tested alone with three small mock modules and a headless simulator that runs much faster than real time. No phone, board or real server is needed.

8.1 Mocks

Mock	Replaces	What it does
Mock physics	M3	Simple bicycle model: turns bot controls into motion with a top speed, acceleration and turn radius
Mock game	M1	Gives packages, marks deliveries, places obstacles from a scenario file
Scripted players	Human players	Motorcycles that follow fixed paths, to test how NPCs react to players

8.2 Test cases

ID	Test	Method	Pass criterion
T1	Graph build	Load the city plan	36 nodes, 60 segments, 120 directed edges, IDs match the formulas
T2	Shortest path	1,000 random obstacle layouts; compare Dijkstra with Floyd–Warshall	Same cost every time
T3	Closed roads	Close roads until a store is cut off	Returns `null`, no crash, no endless loop
T4	Travel-time update	Change an obstacle	New table within the same tick; version number increases
T5	NPC safety	1 hour simulated, 80 cars	No two cars overlap in a lane; no car on a closed road
T6	No deadlock	Same run as T5	No car stopped for more than 10 s without being respawned
T7	Bot delivery	200 delivery cycles from each store, random obstacles	At least 95% completed within 2× the predicted travel time

ID	Test	Method	Pass criterion
T8	Determinism	Run the same scenario twice with the same seed	Identical output hash
T9	Bad input	Missing fields, wrong types, unknown keys	Clear error message, previous state kept, no crash
T10	Performance	80 NPCs and 8 bots	99th-percentile tick time under 2 ms on the main computer

All tests run with one command (`npm test`) and in the group's Git CI on every push. The debug view (Section 5.2) is used for visual checks and for demos of M4 alone.

8.3 Record and replay

M4 can write every `TickInput` to a JSON Lines file. When a bug appears in an integrated game, the file is replayed into M4 alone and the bug appears again, tick by tick.

9. Integration with other modules

M4 joins the system in a fixed order, from the module it depends on most to the least. Each step has one test with a clear pass criterion.

Order	Module	What is exchanged	Integration test	Pass criterion
1	M1 Game Server	Tick input; bot controls; travel-time table	Real game loop with 4 bots, no humans, 5-minute game	Game finishes; every team scores; no exception in the log
2	M1 delivery selection	Travel-time table	100 delivery points chosen for each team	Mean travel time of each team within ±10% of the others
3	M3 Physics	Bot controls in; NPC boxes out; collision events back	Bots drive with real physics; scripted crash into an NPC	Bots still finish ⩾ 90% of deliveries; NPC stops 2 s after a hit
4	M5 Player Client	NPC list through M2	Watch NPC cars on a phone at 30 Hz	No jump larger than 1 m between two updates

Order	Module	What is exchanged	Integration test	Pass criterion
5	M6 Admin Panel	Parameters in; metrics out	Change `npc.count` and `bot.skill` during a game	Change visible within 1 s; metrics graph updates at 2 Hz
6	M8 LED Board (indirect)	Heavy-traffic roads seen on the board	Admin adds heavy traffic on one road	More, slower NPCs on that road in 3D while the board shows it in dim red

Two rules protect the integration:

- The contract files (`city_plan.json` schema and the message schemas in Section 4) live in one shared Git folder. A change needs approval from every module that uses it and increases the version number.
- M1 checks the M4 version at start-up. If the city plan version used by M4 and by the board differ, the game does not start and the admin panel shows why.

10. Performance budget, risks and mitigations

M4 may use at most 3 ms of each 33 ms server tick (30 Hz). The expected cost is under 1 ms: 80 cars × one IDM step, 8 bots × one pure-pursuit step, and Dijkstra only when obstacles change.

Risk	Likelihood	Impact	Mitigation
Server language changes after M4 is written	Medium	Medium	Pure library with JSON contract; no Node-specific code inside the algorithms; port estimated at 1 week
NPC traffic jams at intersections	Medium	High	One-car-per-intersection rule plus 10 s stuck timer (test T6)
Bots get stuck on walls or cars	Medium	Medium	Recover state, replan, and a `bot_stuck` event so M1 can reset the bot
Bots too strong or too weak	High	Low	`bot.skill` parameter and calibration step 3
Unfair delivery distances	Medium	High	Travel-time calibration (step 1) and integration test 2

Risk	Likelihood	Impact	Mitigation
M3 physics not ready when M4 needs it	Medium	Medium	Mock physics (Section 8.1) lets all bot work continue
City plan changes late	Low	High	Versioned file; graph is generated from it, nothing is hard-coded

11. Work plan and open decisions

The work runs in five steps; each ends with something that can be shown alone. Week numbers count from the approval of this proposal and will be matched to the course deadlines.

1. **Weeks 1–2:** City plan loader, road graph, Dijkstra, travel-time table, tests T1–T4. Demo: travel times printed for all four stores.
2. **Weeks 3–4:** NPC traffic with IDM, spawner, intersection rule, debug view, tests T5–T6. Demo: 40 cars driving in the debug view.
3. **Weeks 5–6:** Bot state machine and pure pursuit with mock physics, test T7. Demo: 4 bots playing a full game alone.
4. **Weeks 7–8:** Integration steps 1–3 with M1 and M3; calibration steps 1–2.
5. **Weeks 9–10:** Integration steps 4–6, calibration steps 3–4, performance test T10, final report data.

Proposed work split (to be confirmed by the members): Hasan Alperen Elmas leads routes and travel times, Batuhan Yanar leads NPC traffic and the M6 metrics link, Muhib Taha Boy leads bots and the test harness. Each person reviews another person's code.

Decisions needed from the whole group before submission

- [ ] Server language (M4 recommends TypeScript on Node.js)
- [ ] Server tick rate (M4 recommends 30 Hz)
- [ ] Control message format in Section 3 (shared with M2, M3, M5)
- [ ] Lanes per direction (M4 recommends 2, needed for "accident closes one lane")
- [ ] World size in metres (`size_m`, owned by M3 and M5)
- [ ] Owner of the shared `city_plan.json` file (M4 suggests M1, with M8 as reviewer)